Übungsblatt 2: Scheduling-Theorie und kooperativer Scheduler

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Zielsetzung

Sie prüfen ein Task-Set mit der Liu-und-Layland-Schranke und implementieren auf dem **NUCLEO-H755ZI-Q** (MCU STM32H755ZIT6, Cortex-M4) einen kooperativen Round-Robin-**Dispatcher**. Tasks sind endliche Schritte und kehren zurück. Es gibt keinen Assembler-Kontextwechsel und keine rekursiven Task-Aufrufe. Eine Messung am Oszilloskop ist optional und wird erst mit Protokoll als Messwert gewertet.

Aufgabe 1: Scheduling-Theorie (RMS-Schranke)

Ein Steuergerät soll drei periodische Tasks auf **einer** CPU verarbeiten (implicit deadlines $D_i = T_i$, Annahmen wie in der Vorlesung: unabhängig, voll präemptierbar, kein Overhead):

- τ_1: $C_1 = 1$ ms, $T_1 = 4$ ms
- τ_2: $C_2 = 2$ ms, $T_2 = 6$ ms
- τ_3: $C_3 = 1$ ms, $T_3 = 12$ ms

1. Berechnen Sie U.
2. Vergeben Sie RMS-Prioritäten.
3. Prüfen Sie $U \leq n(2^{1/n} - 1)$. Was folgt, wenn die Schranke **nicht** erfüllt wäre?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 1

$$U = \frac{1}{4} + \frac{2}{6} + \frac{1}{12} = \frac{3}{12} + \frac{4}{12} + \frac{1}{12} = \frac{2}{3} \approx 66{,}7\,\%.$$

Priorität: τ_1 ($T = 4$) > τ_2 ($T = 6$) > τ_3 ($T = 12$).

Für $n = 3$ ist $U_{LL} \approx 77{,}98\,\%$. Da $U < U_{LL}$, ist das Set unter den Liu-Layland-Annahmen hinreichend schedulbar. Wäre U größer als die Schranke, aber ≤ 1, wäre das **keine** Widerlegung — dann Response-Time Analysis. $U > 1$ schließt dauerhafte Pflichtlast auf einer CPU aus.

Aufgabe 2: TCB für endliche Schritte

Definieren Sie im CM4-Projekt (`USER CODE`) einen TCB **ohne** Stackpointer: Funktionszeiger auf einen Step, Zustand, `wake_at`.

1. `TaskStep_t` als Zeiger auf `void (*)(void)`.
2. `struct MyTCB_t` mit `step`, `state`, `wake_at`.
3. `task_list[3]`, `rr_cursor`, `running_idx`.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 2

```c
/* USER CODE BEGIN 0 */
#define N 3
typedef enum { ST_READY = 1, ST_BLOCKED = 0 } TaskState;
typedef void (*TaskStep_t)(void);

typedef struct {
    TaskStep_t step;
    TaskState state;
    uint32_t wake_at;
} MyTCB_t;

MyTCB_t task_list[N];
uint8_t rr_cursor = 0;
uint8_t running_idx = 0;
/* USER CODE END 0 */
```

Aufgabe 3: Dispatcher statt Rekursion

Implementieren Sie `dispatcher()` und `os_delay_ms()`. Der Dispatcher aktualisiert Timeouts, prüft höchstens N Einträge, ruft **einen** bereiten Step auf und bekommt die Kontrolle zurück. Danach wandert der Ringcursor. Ist niemand Ready: Idle (z. B. `__WFI()`), aber nur wenn eine Wake-Quelle läuft (HAL-Tick). Keine `do/while`-Suche ohne Abbruch.

`os_delay_ms` setzt `wake_at` und `ST_BLOCKED` und verlangt, dass der Step **sofort zurückkehrt**. Vergleich mit Wrap: `(int32_t)(now - wake_at) >= 0` für Abstände unter 2^{31} Ticks.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 3

```
/* USER CODE BEGIN 4 */
static void wake_ready(uint32_t now)
{
    for (uint8_t i = 0; i < N; i++) {
        if (task_list[i].state == ST_BLOCKED &&
            (int32_t)(now - task_list[i].wake_at) >= 0) {
            task_list[i].state = ST_READY;
        }
    }
}

void os_delay_ms(uint32_t ms)
{
    task_list[running_idx].wake_at = HAL_GetTick() + ms;
    task_list[running_idx].state = ST_BLOCKED;
    /* Step muss danach returnen - kein Warten hier. */
}

void dispatcher(void)
{
    for (;;) {
        uint32_t now = HAL_GetTick();
        wake_ready(now);
        int sel = -1;
        for (uint8_t k = 0; k < N; k++) {
            uint8_t i = (uint8_t)((rr_cursor + k) % N);
            if (task_list[i].state == ST_READY) {
                sel = (int)i;
                break;
            }
        }
        if (sel < 0) {
            __WFI(); /* SysTick/HAL muss aufwecken können */
            continue;
        }
        rr_cursor = (uint8_t)((sel + 1) % N);
        running_idx = (uint8_t)sel;
        task_list[sel].step();
    }
}
/* USER CODE END 4 */
```

`HAL_GetTick()` braucht eine laufende HAL-Zeitbasis. Der Tick wechselt die Steps nicht präemptiv.

Aufgabe 4: Steps und zwei Messgrößen

Schreiben Sie `Task1_step`, `Task2_step`, `Task3_step`. Jeder Step arbeitet endlich und kehrt zurück (am Ende `os_delay_ms` nur mit sofortigem `return`).

1. `Task1_step`: Pin High, kurze Arbeit, Pin Low. Pin erst nach Belegung wählen (z. B. PA0).
2. In `main` die drei TCBs auf `ST_READY` setzen und `dispatcher()` aufrufen — nicht den Step aus einem anderen Step.
3. Am Oszilloskop **Pulsbreite** w und **Startabstand** P getrennt notieren. Eine konstante Verlängerung von Task 2 ändert P konstant und ist noch kein Jitter. Jitter entsteht bei **wechselnder** Last. Lehrwerte aus der Vorlesung nicht als Ihre Messung eintragen.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 4

```c
void Task1_step(void)
{
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_SET);
    /* endliche Arbeit */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_RESET);
    /* READY lassen und return; os_delay_ms nur bei echtem Schlaf */
}

void Task2_step(void) { /* endliche Arbeit */ }
void Task3_step(void) { /* endliche Arbeit */ }
```

Für periodisches Schlafen: `os_delay_ms(10); return;` — der Dispatcher ruft den Step erst nach `wake_at` wieder auf.

Messhypothese (kein Ersatz für eine Kurve): Pulsbreite ≈ Dauer von Task 1 inklusive zufälliger IRQs. Startabstand hängt von den anderen Step-Dauern ab. Konstante Last ⇒ konstanter Abstand; variable Last ⇒ Peak-to-Peak der Abstände. Hardwareergebnisse nur mit Zeitbasis, Samplezahl und Pin dokumentieren.

Aufgabe 5: Zwei verschiedene Fehler

1. **Kooperativ:** In `Task2_step` eine Endlosschleife ohne `return`. Beobachten Sie Task 1. Aktive Interrupts können weiterlaufen.
2. **Gedankenmodell präemptiv:** High bleibt dauerhaft Ready. Erklären Sie, warum ein `yield` von High, das High sofort wieder Ready lässt, Low nicht aus der Starvation holt.
3. Warum behebt „einfach FreeRTOS“ das nicht automatisch?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 5

Ohne Rückkehr aus Task 2 ruft der Dispatcher Task 1 und Task 3 nicht mehr auf. IRQs (z. B. SysTick) können trotzdem laufen, weil sie den Step präemptieren können — die **Tasks** aber nicht.

Im strikt priorisierten präemptiven Modell wählt der Scheduler nach dem `yield` erneut High, solange High Ready ist. Low braucht, dass High blockiert, ein Budget erschöpft ist oder die Prioritäten anders belegt sind. Präemption garantiert weder Fairness noch eine Frist. FreeRTOS liefert Zustände und Prioritäten; $R \leq D$ bleibt eine Analyse unter Annahmen.